NASCOMSOFT EMBEDDED LTD

COURSE REGISTRATION & ENROLLMENT MANAGEMENT SYSTEM

Complete Implementation Workbook
Code-along guide · VS Code · FastAPI · HTML/CSS · CRUD · GitHub · Deployment

WORKSHOP PROMISE Every development step tells you what file to create or change, immediately shows the exact code or command, and then explains how it works. Follow steps in order; do not skip the checkpoints.

AUDIENCE	SCOPE	DELIVERY
Beginners with Python + FastAPI basics	No database, accounts, JWT or Pydantic models	Run locally → GitHub → deploy demo

One project. One backend. One simple browser interface. Students will create sample courses, enroll fictional students, implement CRUD, handle errors, and deploy a demonstration.

SAFETY / PRIVACY Do not enter real students' details. This app has no login or persistent database; enrollment records are erased by process restarts and are not appropriate for production use.

NASCOMSOFT · WEEK 2 APPLIED BACKEND PROJECT

How to use this workbook

Open VS Code and keep this guide nearby. Each step follows a repeatable pattern: action → exact code/command → explanation → expected result. The major files are assembled incrementally. When a step says CREATE, make that file; when it says APPEND, paste the snippet below the existing content; when it says REPLACE, select all existing content in that file and paste the new block.

Commands are shown for Windows PowerShell and macOS/Linux where the syntax differs. The Python code is identical across platforms. Save files before refreshing the browser. Local URLs are shown using the default port 8000.

What you will build

COMPONENT	JOB
FastAPI backend	Expose JSON API routes for course catalog and enrollment CRUD.
Temporary Python storage	Keep courses and enrollments in simple lists of dictionaries.
HTML template	Display registration form, courses, enrollment table, filter and edit/delete forms.
CSS stylesheet	Give the single-page dashboard a consistent Nascomsoft-inspired appearance.
GitHub repository	Record work and share the source.
Python web deployment	Run the same project online for demonstration.

Prerequisites & boundaries

Students need Python 3.10 or newer, VS Code, a terminal, Git, a browser, and internet access for installation and deployment. An account on GitHub is needed for the publishing phase. No JavaScript or database is required. FastAPI uses Pydantic internally, but this workshop does not introduce Pydantic model classes; request bodies use simple dictionaries.

IN-MEMORY RULE If the server process restarts, enrollment records disappear and the next ID starts again at 101. Course records remain because they are defined in the Python source file.

Build order at a glance

PHASE	IMPLEMENTATION OUTPUT
1 · Plan	Agree on user journey, business rules, endpoints and MVP.
2 · Setup	Create project, virtual environment, requirements and first FastAPI route.
3 · Data	Model courses/enrollments and basic validation/CRUD functions.
4 · Read	Implement GET courses, GET enrollment and query filtering.
5 · Write	Implement POST, PUT and DELETE with appropriate status codes.
6 · Browser	Create HTML + CSS; connect forms to FastAPI.
7 · Test	Run real scenarios locally, including invalid inputs and missing IDs.
8 · GitHub	Publish repository with useful README and .gitignore.
9 · Deployment	Connect GitHub to a Python host and demonstrate a public URL.

PHASE 1 · Plan before coding

We keep the problem small: a Nascomsoft staff member enrolls a fictional student into one illustrative training course.

Step 1 — Define the classroom user journey

Scenario: An administrator opens the dashboard, selects one course, enters a name/email/phone, presses Register, sees the new enrollment in a table, and can later edit or delete it. The application also exposes API endpoints for testing through /docs.

SCOPE CONTRACT Must have: course list, enrollment create/read/update/delete, course filter, meaningful 400/404 responses, local browser interface, GitHub repository, and deployment attempt. Must not have: accounts, payment, databases, advanced JavaScript or real student data.

Step 2 — Agree on the resources & routes

METHOD	ENDPOINT	PURPOSE
GET	/courses	Show illustrative courses.
GET	/courses/{course_id}	Show a single course.
GET	/enrollments	Show all registrations.
GET	/enrollments/{enrollment_id}	Find one registration.
GET	/enrollments?course_id=2	Filter registrations by course.
POST	/enrollments	Create enrollment (201).
PUT	/enrollments/{enrollment_id}	Replace enrollment details.
DELETE	/enrollments/{enrollment_id}	Remove enrollment (204).

A course is a training program offered in the demo. An enrollment is one registration for one course. Course IDs link the records. If an enrollment or course is missing, the backend should return 404.

PHASE 2 · Project setup & first server

These are the precise steps students should perform in VS Code before writing the main system.

Step 3 — Create the project folder

Open a terminal and create the root project directory. Run the appropriate command while in a location where you want the project stored.

TERMINAL

```
mkdir nascomsoft-course-registration
cd nascomsoft-course-registration
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

mkdir: Creates the new project directory.

cd: Moves the terminal into that directory. All later commands must run here.

CHECKPOINT Your terminal path ends with nascomsoft-course-registration.

Step 4 — Open the project in VS Code

Open the current directory in VS Code using the terminal if the code launcher is installed.

TERMINAL

```
code .
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

code: Starts Visual Studio Code from the terminal.

dot (.): Means the directory you are currently in. If the command is unavailable, use File → Open Folder in VS Code.

Step 5 — Create the folder structure

Use the Explorer panel to create templates and static. Create the files below as they are introduced; do not paste every source file at once.

FOLDER TREE

```
nascomsoft-course-registration/
├── main.py
├── data_store.py
├── requirements.txt
├── README.md
├── .gitignore
├── templates/
│   └── index.html
└── static/
    └── styles.css
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

main.py: FastAPI app, API routes and form-handling routes.

data_store.py: Sample course records and functions that perform CRUD operations.

templates/index.html: HTML dashboard, registration form and enrollment table.

static/styles.css: CSS rules that style the page.

requirements.txt: Third-party Python packages to install.

README.md and .gitignore: Project instructions and rules for files Git should ignore.

Step 6 — Create a virtual environment

Still inside the root folder, create an isolated Python environment.

TERMINAL

```
python -m venv .venv
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

python: Runs your installed Python interpreter. On systems that use python3, replace python with python3.

-m venv: Runs Python’s standard virtual-environment module.

.venv: Names the local environment folder; do not commit it to GitHub.

Step 7 — Activate your virtual environment

Run the command for your operating system in a VS Code terminal.

TERMINAL — CHOOSE ONE

```
# Windows PowerShell
.venv\Scripts\Activate.ps1

# Windows Command Prompt
.venv\Scripts□ctivate.bat

# macOS / Linux
source .venv/bin/activate
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

Windows PowerShell: Runs the PowerShell activation script. If blocked by execution policy, use Command Prompt or an approved local PowerShell policy change.

Windows Command Prompt: Runs the batch-file version of the activation script.

macOS/Linux: Uses the shell source command. The prompt commonly changes to show (.venv).

CHECKPOINT Your active terminal usually shows (.venv) at the beginning of its prompt.

Step 8 — Create requirements.txt

In VS Code, create requirements.txt at the root and paste all four lines.

REQUIREMENTS.TXT — CREATE

```
fastapi>=0.115,<1.0
uvicorn[standard]>=0.30,<1.0
jinja2>=3.1,<4.0
python-multipart>=0.0.9,<1.0
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

fastapi: Provides route decorators, request/response handling and interactive documentation.

uvicorn[standard]: Runs the ASGI web server locally and in deployment.

jinja2: Fills HTML templates with Python data supplied by FastAPI.

python-multipart: Allows FastAPI to read values submitted by standard HTML forms.

version ranges: Permit compatible updates; installing into the virtual environment keeps project packages isolated.

Step 9 — Install the dependencies

From the root folder with your virtual environment active, install the packages listed in requirements.txt.

TERMINAL

```
python -m pip install -r requirements.txt
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

python -m pip: Ensures that the pip installer belongs to the Python interpreter you are using.

-r requirements.txt: Tells pip to install every dependency listed in the file.

CHECKPOINT The terminal reports successful installation, without ModuleNotFoundError.

Step 10 — Create the very first main.py

Create main.py in the root directory. This is a temporary first version; we will replace it with the real project foundation later.

PYTHON

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/hello")
def hello():
    return {"message": "Nascomsoft FastAPI is running"}
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

Line 1: Imports the FastAPI class from the installed fastapi package.

Line 3: Creates the web application object named app.

Line 6: Registers the HTTP GET /hello endpoint.

Line 7: Defines the Python function FastAPI will call for that route.

Line 8: Returns a dictionary. FastAPI converts it into JSON for the HTTP response.

Step 11 — Run Uvicorn and open the browser

Run this while you are still in the root folder. Leave the terminal running.

TERMINAL

```
python -m uvicorn main:app --reload
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

python -m uvicorn: Starts the server from the active Python environment.

main:app: Means “find app inside the main.py module.”

--reload: Watches files and restarts the local development server when Python code changes. Do not use this flag in production.

CHECKPOINT Open http://127.0.0.1:8000/hello and expect {"message":"Nascomsoft FastAPI is running"}. Also open http://127.0.0.1:8000/docs.

PHASE 3 · Store courses and registrations

We now model the workshop data using ordinary Python lists and dictionaries. The functions below separate business rules from HTTP routing.

Step 12 — Create data_store.py with sample courses

Create data_store.py. This block is the first part of the file. It defines five illustrative courses, an empty enrollment list and the initial ID counter.

DATA_STORE.PY — CREATE · PART 1 OF 4

```
"""Small in-memory data layer for the Nascomsoft training demo."""

COURSES = [
    {
        "course_id": 1,
        "title": "Python Programming",
        "duration": "4 Weeks",
        "description": "Programming foundations with Python.",
    },
    {
        "course_id": 2,
        "title": "Software Development with Python",
        "duration": "4 Weeks",
        "description": "Build web APIs with FastAPI.",
    },
    {
        "course_id": 3,
        "title": "Embedded Systems Fundamentals",
        "duration": "4 Weeks",
        "description": "An introduction to embedded applications.",
    },
    {
        "course_id": 4,
        "title": "Web Development",
        "duration": "4 Weeks",
        "description": "Build the structure and style of web pages.",
    },
    {
        "course_id": 5,
        "title": "Data Analysis with Python",
        "duration": "4 Weeks",
        "description": "Explore and summarize data in Python.",
    },
]

ENROLLMENTS = []
NEXT_ENROLLMENT_ID = 101
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

Docstring: Describes the role of the file; it is informational text, not executable business logic.

COURSES = [...]: A list containing five course dictionaries. Each dictionary has course_id, title, duration and description. These are demo offerings, not a verified course catalog.

course_id: A unique number used to identify a particular course in requests and enrollment records.

title, duration and description: Human-readable details that the browser can display on the course cards.

ENROLLMENTS = []: An empty list that will store newly created enrollment dictionaries while the process runs.

NEXT_ENROLLMENT_ID = 101: The next number assigned to a new enrollment. Starting at 101 makes IDs easy to notice in the classroom.

Step 13 — APPEND lookup and filtering functions

Scroll to the bottom of data_store.py and paste the next block beneath what you already wrote.

DATA_STORE.PY — APPEND · PART 2 OF 4

```
def find_course(course_id):
    for course in COURSES:
        if course["course_id"] == course_id:
            return course
    return None


def find_enrollment(enrollment_id):
    for record in ENROLLMENTS:
        if record["enrollment_id"] == enrollment_id:
            return record
    return None


def list_enrollments(course_id=None):
    if course_id is None:
        return ENROLLMENTS
    return [r for r in ENROLLMENTS if r["course_id"] == course_id]
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

find_course(course_id): Searches COURSES for the matching course_id.

for course in COURSES: Checks each course dictionary one after another.

if ... == course_id: Tests whether the stored identifier matches the function argument.

return course: Stops the search and gives the caller the matching dictionary.

return None: Signals that no matching record exists; routes later translate that into an HTTP 404.

find_enrollment: Performs the same lookup pattern inside ENROLLMENTS.

list_enrollments(course_id=None): Returns everything when no filter was requested, otherwise uses a list comprehension to keep only matching course IDs.

Step 14 — APPEND validation rules

At the bottom of data_store.py, paste this validation function. It checks the incoming registration before storing it.

DATA_STORE.PY — APPEND · PART 3 OF 4

```
def validate_enrollment_data(data):
    if not isinstance(data, dict):
        raise ValueError("Registration data must be a JSON object.")

    name = data.get("student_name")
    email = data.get("email")
    phone = data.get("phone")
    course_id = data.get("course_id")

    if not isinstance(name, str) or not name.strip():
        raise ValueError("Student name is required.")
    if not isinstance(email, str) or "@" not in email or not email.strip():
        raise ValueError("Enter a valid-looking email address.")
    if not isinstance(phone, str) or not phone.strip():
        raise ValueError("Phone number is required.")
    if type(course_id) is not int or find_course(course_id) is None:
        raise ValueError("Choose an existing course ID.")

    return {
        "student_name": name.strip(),
        "email": email.strip(),
        "phone": phone.strip(),
        "course_id": course_id,
    }
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

def validate_enrollment_data(data): Creates one reusable function for both the JSON API and the HTML forms.

isinstance(data, dict): Rejects values that are not dictionary-shaped registration objects.

data.get(...): Reads keys safely; returns None if a key is missing rather than crashing with KeyError.

student_name check: Requires a non-empty string after removing spaces.

email check: Performs only a basic classroom “contains @” check; it is NOT complete production email validation.

phone check: Requires a non-empty string and does not assume a specific national phone format.

type(course_id) is not int: Makes sure course_id is an integer (not a string or Boolean).

find_course(course_id) is None: Rejects IDs that do not refer to an illustrative course.

return {...}: Returns a cleaned dictionary containing the four recognized fields; name/email/phone are stripped of surrounding spaces.

Step 15 — APPEND create, update and delete functions

This final block completes data_store.py. It is the heart of our CRUD operations.

DATA_STORE.PY — APPEND · PART 4 OF 4

```
def add_enrollment(data):
    global NEXT_ENROLLMENT_ID
    fields = validate_enrollment_data(data)
    record = {"enrollment_id": NEXT_ENROLLMENT_ID, **fields}
    ENROLLMENTS.append(record)
    NEXT_ENROLLMENT_ID += 1
    return record


def update_enrollment(enrollment_id, data):
    record = find_enrollment(enrollment_id)
    if record is None:
        raise LookupError("Enrollment not found.")
    fields = validate_enrollment_data(data)
    record.update(fields)
    return record


def remove_enrollment(enrollment_id):
    record = find_enrollment(enrollment_id)
    if record is None:
        raise LookupError("Enrollment not found.")
    ENROLLMENTS.remove(record)
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

add_enrollment(data): Implements CREATE: validates fields, constructs a record, appends it to the list, increases the next-ID counter and returns the new record.

global NEXT_ENROLLMENT_ID: Allows the function to change the module-level ID counter.

**fields: Inserts the cleaned key-value pairs into the new enrollment dictionary.

ENROLLMENTS.append(record): Stores the dictionary in the shared in-memory list.

update_enrollment(id, data): Implements UPDATE: locate the old record, reject missing IDs, validate new values, and modify the original dictionary.

record.update(fields): Changes the values of the existing dictionary without changing the enrollment ID.

remove_enrollment(id): Implements DELETE: locate the record, reject missing IDs, and remove the dictionary from the list.

LookupError vs ValueError: LookupError means the requested enrollment is missing; ValueError means submitted registration details are invalid.

CHECKPOINT Save data_store.py. Do not start a second Python program for it: main.py will import these functions.

PHASE 4 · Build read-only FastAPI endpoints

Replace the temporary hello application with the real project foundation. Then append the GET endpoints. This builds READ before WRITE.

Step 16 — REPLACE main.py with the project foundation

Open main.py, select all and replace its contents with the block below. Later API/UI blocks will be APPENDED beneath it.

MAIN.PY — REPLACE · FOUNDATION

```
"""FastAPI endpoints and HTML forms for the Nascomsoft classroom project."""

from urllib.parse import urlencode

from fastapi import Body, FastAPI, Form, HTTPException, Request, Response
from fastapi.responses import HTMLResponse, RedirectResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

from data_store import (
    COURSES,
    add_enrollment,
    find_course,
    find_enrollment,
    list_enrollments,
    remove_enrollment,
    update_enrollment,
)

app = FastAPI(title="Nascomsoft Course Registration API", version="1.0.0")
app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

urlencode: Safely builds query strings used for browser success/error messages.

Body, FastAPI, Form, HTTPException: FastAPI utilities for JSON request bodies, forms, errors and app creation.

Request, Response: Represent incoming HTTP requests and outgoing responses.

HTMLResponse, RedirectResponse: Tell FastAPI to serve HTML or redirect a browser after submitting a form.

StaticFiles: Serves the CSS file from the static folder.

Jinja2Templates: Loads an HTML file and fills placeholders with data.

from data_store import (...): Imports the data and CRUD functions students just created, so backend routes can reuse the same rules.

app = FastAPI(...): Creates an app with a descriptive name and version displayed on /docs.

app.mount(...): Makes styles.css accessible at /static/styles.css.

templates = Jinja2Templates(...): Sets templates/ as the place to find index.html.

Step 17 — APPEND health and course catalog routes

Paste these routes below the last line in main.py. The server can now answer health checks and requests about available courses.

MAIN.PY — APPEND · HEALTH & COURSES

```
@app.get("/health")
def health_check():
    return {"status": "ok"}


@app.get("/courses")
def get_courses():
    return COURSES


@app.get("/courses/{course_id}")
def get_course(course_id: int):
    course = find_course(course_id)
    if course is None:
        raise HTTPException(status_code=404, detail="Course not found.")
    return course
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

@app.get("/health"): Declares an easy endpoint to verify that the deployed process is running.

health_check and return: Returns a small JSON object with the text status: ok.

@app.get("/courses"): Declares the URL for reading the entire course catalog.

return COURSES: FastAPI converts the Python list into a JSON array.

@app.get("/courses/{course_id}"): Declares a dynamic path. 2 in /courses/2 becomes the course_id argument.

course_id: int: Asks FastAPI to parse the URL segment as an integer.

find_course: Delegates the search to our data_store function.

HTTPException(404): Returns a controlled “Course not found” response instead of a Python crash.

CHECKPOINT At /docs, expand GET /courses; execute it and expect five courses. GET /courses/2 should succeed; GET /courses/99 should return 404.

Step 18 — APPEND enrollment READ routes

Paste the next GET routes beneath the course routes in main.py.

MAIN.PY — APPEND · READ ENROLLMENTS

```
@app.get("/enrollments")
def get_enrollments(course_id: int | None = None):
    if course_id is not None and find_course(course_id) is None:
        raise HTTPException(status_code=404, detail="Course not found.")
    return list_enrollments(course_id)


@app.get("/enrollments/{enrollment_id}")
def get_enrollment(enrollment_id: int):
    record = find_enrollment(enrollment_id)
    if record is None:
        raise HTTPException(status_code=404, detail="Enrollment not found.")
    return record
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

GET /enrollments: Reads all enrollment records.

course_id: int | None = None: Makes course_id an optional query parameter; omit it for all courses or supply ?course_id=2 to filter.

course exists check: Avoids silently treating an unknown course ID as a valid empty result.

list_enrollments(course_id): Returns either the full list or the filtered list.

GET /enrollments/{enrollment_id}: Reads one enrollment using a path parameter.

find_enrollment: Searches for that ID; None means there is no such record.

404 if missing: Communicates a missing enrollment to the HTTP client.

return record: Returns the matching dictionary as JSON.

CHECKPOINT Because no students have been registered yet, GET /enrollments should return []. GET /enrollments/101 currently returns 404.

PHASE 5 · Implement CREATE, UPDATE and DELETE

In this phase, the API can change stored data. Each HTTP method maps to the corresponding CRUD operation. These routes accept plain dictionaries; no Pydantic model classes are defined.

CRUD map: what each operation means

CRUD	HTTP	APPLICATION BEHAVIOR	EXPECTED SUCCESS
CREATE	POST	Add one enrollment	201 Created
READ	GET	Get list or one record	200 OK
UPDATE	PUT	Replace enrollment fields	200 OK
DELETE	DELETE	Remove an enrollment	204 No Content

Step 19 — APPEND POST to create registrations

Below the existing GET routes, add this block.

MAIN.PY — APPEND · CREATE

```
@app.post("/enrollments", status_code=201)
def create_enrollment(payload: dict = Body(...)):
    try:
        return add_enrollment(payload)
    except ValueError as error:
        raise HTTPException(status_code=400, detail=str(error)) from error
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

@app.post("/enrollments", status_code=201): Registers a POST endpoint and instructs FastAPI to return 201 on success.

payload: dict = Body(...): Tells FastAPI to read a required JSON object from the request body; no separate Pydantic class is introduced.

try / add_enrollment: Calls the existing validation and storage function and returns the newly created record.

except ValueError: Catches invalid student details or an unknown course ID.

HTTPException(status_code=400): Turns that application error into a useful HTTP 400 response with the message.

Step 20 — Create your first enrollment in /docs

Open http://127.0.0.1:8000/docs, expand POST /enrollments, click Try it out, replace the example body with this JSON, then Execute.

JSON — REQUEST BODY

```
{
  "student_name": "Ada James",
  "email": "ada@example.com",
  "phone": "08012345678",
  "course_id": 2
}
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

Braces { ... }: Define one JSON object; it becomes a Python dictionary for our endpoint.

student_name, email, phone: Must be non-empty strings.

course_id: 2: A JSON number, not the string "2"; it refers to Software Development with Python.

Response: Expect HTTP 201 with enrollment_id 101 plus the other submitted fields.

CHECKPOINT Refresh GET /enrollments; it should now return a list containing Ada’s record.

Step 21 — APPEND PUT and DELETE routes

Add these two endpoint functions at the bottom of main.py.

MAIN.PY — APPEND · UPDATE & DELETE

```
@app.put("/enrollments/{enrollment_id}")
def replace_enrollment(enrollment_id: int, payload: dict = Body(...)):
    try:
        return update_enrollment(enrollment_id, payload)
    except LookupError as error:
        raise HTTPException(status_code=404, detail=str(error)) from error
    except ValueError as error:
        raise HTTPException(status_code=400, detail=str(error)) from error


@app.delete("/enrollments/{enrollment_id}", status_code=204)
def delete_enrollment(enrollment_id: int):
    try:
        remove_enrollment(enrollment_id)
    except LookupError as error:
        raise HTTPException(status_code=404, detail=str(error)) from error
    return Response(status_code=204)
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

PUT /enrollments/{enrollment_id}: Targets one existing registration for full replacement of editable fields.

payload: dict = Body(...): Expects the complete set of registration fields as JSON, not merely one changed field.

update_enrollment: Reuses the business-rule helper to find, validate and modify the record.

except LookupError → 404: Returns Not Found when the target ID does not exist.

except ValueError → 400: Returns Bad Request when the replacement data violates our rules.

DELETE /enrollments/{enrollment_id}: Targets one registration for removal.

remove_enrollment: Deletes the record from the shared list.

status_code=204 and Response(204): A successful DELETE returns no response body; Swagger shows 204 No Content.

CHECKPOINT Try PUT /enrollments/101 with a changed name and all fields; then GET /enrollments/101. Try DELETE /enrollments/101 and verify a later GET returns 404.

PHASE 6 · Build the browser interface

Next we add a small server-rendered HTML page. This is a deliberate teaching shortcut: no JavaScript framework is needed. HTML forms use POST, while our JSON API still exposes proper PUT and DELETE.

Step 22 — CREATE templates/index.html — document shell

Create templates/index.html and paste this opening portion. Do not close the file yet: subsequent steps append page sections.

INDEX.HTML — CREATE · PART 1 OF 4

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Nascomsoft Course Registration</title>
  <link rel="stylesheet" href="/static/styles.css">
</head>
<body>
  <header class="site-header">
    <h1>NASCOMSOFT EMBEDDED LTD</h1>
    <p>Course Registration &amp; Enrollment Dashboard</p>
  </header>
  <main class="container">
    {% if success %}<p class="notice success">{{ success }}</p>{% endif %}
    {% if error %}<p class="notice error">{{ error }}</p>{% endif %}
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

DOCTYPE / html: Declare a modern HTML document and indicate English as the page language.

head / meta: Set character encoding and make the page responsive on small screens.

title: Controls the browser-tab title.

link rel="stylesheet": Downloads CSS from the /static/styles.css URL served by FastAPI.

body / header: Starts the visible page and identifies Nascomsoft Embedded LTD.

main class="container": Holds the main content inside a centered layout.

{% if success %} / {% if error %}: Jinja2 conditionally displays messages passed from the backend; {{ ... }} inserts values safely into HTML.

Step 23 — APPEND registration and course-catalog sections

Paste the next HTML block below the existing content in index.html.

INDEX.HTML — APPEND · PART 2 OF 4

```
<section class="panel">
  <h2>Register a student</h2>
  <form method="post" action="/ui/register" class="form-grid">
    <label>Full name
      <input type="text" name="student_name" required>
    </label>
    <label>Email
      <input type="email" name="email" required>
    </label>
    <label>Phone
      <input type="tel" name="phone" required>
    </label>
    <label>Course
      <select name="course_id" required>
        {% for course in courses %}
        <option value="{{ course.course_id }}">{{ course.title }}</option>
        {% endfor %}
      </select>
    </label>
    <button type="submit">Register student</button>
  </form>
</section>

<section class="panel">
  <h2>Available courses</h2>
  <div class="course-grid">
    {% for course in courses %}
    <article class="course-card">
      <strong>{{ course.title }}</strong>
      <span>{{ course.duration }}</span>
      <p>{{ course.description }}</p>
    </article>
    {% endfor %}
  </div>
</section>
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

section class="panel": Groups content into separate card-like regions of the page.

form method="post" action="/ui/register": Submits the registration fields to FastAPI using an ordinary HTML POST.

input name="...": Each input name becomes a corresponding Form(...) parameter in the route.

required: Asks the browser to check that a field is non-empty before submitting; backend validation still matters.

select name="course_id": Presents a dropdown of available courses.

{% for course in courses %}: Jinja2 loops through the course dictionaries supplied by FastAPI.

{{ course.course_id }} and {{ course.title }}: Generate the option value and the visible label for each course.

button type="submit": Submits the form to the /ui/register route.

course-grid loop: Displays all sample courses as cards, including course title, duration and description.

Step 24 — APPEND enrollment table and course filter

Continue editing templates/index.html; append this listing and filter block.

INDEX.HTML — APPEND · PART 3 OF 4

```
<section class="panel">
  <h2>Enrollment records</h2>
  <form method="get" action="/" class="filter-row">
    <label>Filter by course
      <select name="course_id">
        <option value="">All courses</option>
        {% for course in courses %}
        <option value="{{ course.course_id }}"
          {% if active_course == course.course_id %}selected{% endif %}>
          {{ course.title }}
        </option>
        {% endfor %}
      </select>
    </label>
    <button type="submit">Apply filter</button>
    <a href="/">Clear</a>
  </form>
  <table>
    <thead>
      <tr><th>ID</th><th>Name</th><th>Email</th><th>Course ID</th></tr>
    </thead>
    <tbody>
      {% for item in enrollments %}
      <tr>
        <td>{{ item.enrollment_id }}</td>
        <td>{{ item.student_name }}</td>
        <td>{{ item.email }}</td>
        <td>{{ item.course_id }}</td>
      </tr>
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

section and h2: Start the part of the page used to manage enrollment records.

form method="get" action="/": Sends a simple browser query such as /?course_id=2 to filter visible records.

All courses option: A blank value means no filter.

active_course == course.course_id: Selects the correct dropdown item when the current page is filtered.

table / thead / tbody: Create column headings and a body area for enrollment rows.

{% for item in enrollments %}: Create one set of rows for each enrollment returned by FastAPI.

{{ item.enrollment_id }} etc.: Display values from each enrollment dictionary as table cells.

Step 25 — APPEND edit/delete forms and close HTML

Paste this final block directly below the table row from the previous step. It adds the editable controls and closes all HTML tags.

INDEX.HTML — APPEND · PART 4 OF 4

```
          <tr class="action-row"><td colspan="4">
            <details>
              <summary>Edit or delete enrollment #{{ item.enrollment_id }}</summary>
              <form method="post"
                    action="/ui/enrollments/{{ item.enrollment_id }}/update"
                    class="form-grid small">
                <label>Name <input name="student_name"
                       value="{{ item.student_name }}" required></label>
                <label>Email <input type="email" name="email"
                       value="{{ item.email }}" required></label>
                <label>Phone <input name="phone"
                       value="{{ item.phone }}" required></label>
                <label>Course
                  <select name="course_id">
                    {% for course in courses %}
                    <option value="{{ course.course_id }}"
                      {% if item.course_id == course.course_id %}selected{% endif %}>
                      {{ course.title }}
                    </option>
                    {% endfor %}
                  </select>
                </label>
                <button type="submit">Save changes</button>
              </form>
              <form method="post"
                    action="/ui/enrollments/{{ item.enrollment_id }}/delete">
                <button class="danger" type="submit">Delete enrollment</button>
              </form>
            </details>
          </td></tr>
          {% else %}
          <tr><td colspan="4">No enrollments to display yet.</td></tr>
          {% endfor %}
        </tbody>
      </table>
    </section>
    <p class="footnote">Classroom demonstration only. Use fictional data.</p>
  </main>
</body>
</html>
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

details / summary: Let users reveal the editing controls for a specific enrollment without JavaScript.

form action="/ui/enrollments/.../update": Sends updated values to the dedicated HTML form-handling POST route.

value="{{ item... }}": Prefills fields with current values so the administrator can edit them.

course dropdown with selected condition: Keeps the current course selected until changed.

Save changes button: Submits the update form to FastAPI.

second form / delete button: Posts an empty form to a dedicated delete handler for that specific enrollment.

{% else %} and {% endfor %}: Show “No enrollments...” if the list is empty, then finish the Jinja2 loop.

closing tags: Finish the table, section, page container, body and HTML document.

CHECKPOINT Save templates/index.html. It is not fully operational until we append the UI routes to main.py.

Step 26 — CREATE static/styles.css

Create static/styles.css and paste this complete stylesheet. CSS affects appearance, not the underlying CRUD logic.

STYLES.CSS — CREATE

```
:root { font-family: Arial, sans-serif; color: #172b3a; background: #f3f6f8; }
* { box-sizing: border-box; }
body { margin: 0; }
.site-header { background: #0b6170; color: white; padding: 25px 6%; }
.site-header h1 { margin: 0; font-size: 25px; letter-spacing: 0.5px; }
.site-header p { margin: 7px 0 0; }
.container { max-width: 1100px; margin: 22px auto; padding: 0 18px; }
.panel { background: white; border: 1px solid #dbe4e8; border-radius: 12px;
         padding: 20px; margin-bottom: 20px; }
h2 { margin-top: 0; color: #0b6170; }
.form-grid { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr));
             gap: 12px; align-items: end; }
label { display: block; font-size: 14px; font-weight: bold; }
input, select { display: block; width: 100%; padding: 10px; margin-top: 5px;
                border: 1px solid #b7c9cf; border-radius: 6px; }
button { background: #0b6170; color: white; border: 0; border-radius: 6px;
         padding: 11px 15px; cursor: pointer; }
button:hover { background: #084c58; }
button.danger { background: #a52d34; margin-top: 10px; }
.course-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(190px, 1fr));
               gap: 12px; }
.course-card { background: #edf6f7; padding: 13px; border-radius: 8px; }
.course-card strong, .course-card span { display: block; }
.course-card span { font-size: 12px; color: #4e6c73; margin-top: 6px; }
.course-card p { font-size: 13px; }
.filter-row { display: flex; gap: 12px; align-items: end; margin-bottom: 15px; }
.filter-row label { min-width: 230px; }
table { width: 100%; border-collapse: collapse; }
th, td { border-bottom: 1px solid #d9e3e6; padding: 11px; text-align: left; }
th { background: #e8f1f3; }
.action-row td { background: #f8fbfc; }
summary { cursor: pointer; color: #0b6170; font-size: 13px; }
.small { margin-top: 13px; }
.notice { padding: 13px; border-radius: 7px; }
.success { background: #e2f5e8; }
.error { background: #ffe6e6; }
.footnote { text-align: center; font-size: 12px; color: #5c737c; }
@media (max-width: 640px) {
  .form-grid { grid-template-columns: 1fr; }
  .filter-row { flex-wrap: wrap; }
  table { font-size: 12px; }
}
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

:root / body / *: Set page typography, base colors, spacing and consistent element sizing.

.site-header / h1 / p: Give the Nascomsoft heading a teal background and readable text.

.container / .panel / h2: Center the content and create white cards with teal section headings.

.form-grid / label / input / select: Lay out and style the student registration and editing forms.

button / .danger: Style standard actions teal and deletion actions red.

.course-grid / .course-card: Arrange sample courses in responsive cards.

table / th / td: Give the enrollment dashboard clear columns and dividing lines.

.notice / .success / .error: Visually distinguish confirmation messages from validation errors.

@media (max-width: 640px): On smaller screens, use one-column forms and allow the filter controls to wrap.

Step 27 — APPEND browser navigation and the dashboard route

Return to main.py. Add this code after the DELETE API function. This joins FastAPI, Jinja2 and the HTML you wrote.

MAIN.PY — APPEND · DASHBOARD PAGE

```
def redirect_to_dashboard(kind, message):
    query = urlencode({kind: message})
    return RedirectResponse(url=f"/?{query}", status_code=303)


@app.get("/", response_class=HTMLResponse, include_in_schema=False)
def dashboard(request: Request, course_id: int | None = None):
    records = list_enrollments(course_id)
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={
            "courses": COURSES,
            "enrollments": records,
            "active_course": course_id,
            "success": request.query_params.get("success", ""),
            "error": request.query_params.get("error", ""),
        },
    )
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

redirect_to_dashboard: Builds a query string with a success/error message then issues HTTP 303 so the browser returns to GET /.

urlencode: Encodes spaces, # and other special characters safely in the URL.

@app.get("/", response_class=HTMLResponse): Uses GET / for the human-facing dashboard, not JSON.

include_in_schema=False: Hides browser-only helper routes from /docs so the REST endpoints remain the focus.

request: Request: Lets us read the incoming browser request and query parameters.

list_enrollments(course_id): Applies the course filter while choosing the records that the HTML template will display.

TemplateResponse request/name/context: Renders templates/index.html and supplies courses, records, filter value and message text as template variables.

CHECKPOINT Visit http://127.0.0.1:8000/. You should see the branded page, course cards and an empty enrollment table. CSS should load.

Step 28 — APPEND the HTML registration handler

Continue at the bottom of main.py. This function receives values from the browser form, translates them into a dictionary, and reuses the same add_enrollment function as the REST API.

MAIN.PY — APPEND · FORM CREATE

```
@app.post("/ui/register", include_in_schema=False)
def register_from_form(
    student_name: str = Form(...),
    email: str = Form(...),
    phone: str = Form(...),
    course_id: int = Form(...),
):
    data = {
        "student_name": student_name,
        "email": email,
        "phone": phone,
        "course_id": course_id,
    }
    try:
        record = add_enrollment(data)
    except ValueError as error:
        return redirect_to_dashboard("error", str(error))
    return redirect_to_dashboard(
        "success", f"Enrollment #{record['enrollment_id']} created."
    )
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

@app.post("/ui/register"): Defines the browser-form route receiving POST requests from HTML.

Form(...) parameters: FastAPI reads each submitted form field using its name attribute in the HTML.

course_id: int: Converts the course dropdown value from form text into an integer.

data = {...}: Builds the same dictionary structure used by POST /enrollments.

add_enrollment(data): Calls the shared CRUD logic, so browser and API create identical records.

except ValueError: Catches a validation failure and redirects with an error message.

success redirect: Reports the created enrollment ID and sends the browser back to the dashboard.

CHECKPOINT Fill in the registration form with fictional details, select course 2 and click Register student. A success banner should appear and the enrollment should be listed.

Step 29 — APPEND browser edit and delete handlers

This last main.py block finishes the user interface. HTML forms cannot send PUT/DELETE methods directly, so these special POST routes call the same underlying functions.

MAIN.PY — APPEND · FORM UPDATE & DELETE

```
@app.post("/ui/enrollments/{enrollment_id}/update", include_in_schema=False)
def update_from_form(
    enrollment_id: int,
    student_name: str = Form(...),
    email: str = Form(...),
    phone: str = Form(...),
    course_id: int = Form(...),
):
    data = {
        "student_name": student_name,
        "email": email,
        "phone": phone,
        "course_id": course_id,
    }
    try:
        update_enrollment(enrollment_id, data)
    except (LookupError, ValueError) as error:
        return redirect_to_dashboard("error", str(error))
    return redirect_to_dashboard("success", "Enrollment updated.")


@app.post("/ui/enrollments/{enrollment_id}/delete", include_in_schema=False)
def delete_from_form(enrollment_id: int):
    try:
        remove_enrollment(enrollment_id)
    except LookupError as error:
        return redirect_to_dashboard("error", str(error))
    return redirect_to_dashboard("success", "Enrollment deleted.")
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

POST /ui/enrollments/{id}/update: Receives an editing form for a specific enrollment ID.

Form(...) fields: Collect the replacement name/email/phone/course values from the HTML controls.

update_enrollment(enrollment_id, data): Runs exactly the same validation and update operation used by the PUT API route.

except (LookupError, ValueError): Handles missing enrollments or invalid inputs by sending the user back to an error banner.

success redirect: Confirms that an existing registration was updated.

POST /ui/enrollments/{id}/delete: Receives the delete button submission from the browser.

remove_enrollment(enrollment_id): Runs the same data deletion helper used by DELETE /enrollments/{id}.

browser redirect: Returns to the dashboard with an informative confirmation banner.

CHECKPOINT Open “Edit or delete enrollment #...” beneath a record, change its name/course and Save changes. Then delete it and confirm it disappears.

PHASE 7 · Test the complete system

A project is not complete merely because the home page opens. Test both API behavior and the browser workflow.

Step 30 — Run a complete API test sequence

Open /docs and follow the sequence below. The expected ID of the first enrollment is 101 if you start from a fresh server process. If you already created sample records, use the ID returned in your current response instead.

NO.	ACTION	EXPECTED RESPONSE
1	GET /health	200 and {"status":"ok"}.
2	GET /courses	200 and five illustrative course objects.
3	GET /enrollments	200 and [] for a fresh process.
4	POST /enrollments (valid JSON)	201; new enrollment with an ID.
5	GET /enrollments/{id}	200; exactly that enrollment.
6	GET /enrollments?course_id=2	200; records whose course_id equals 2.
7	PUT /enrollments/{id} (full JSON)	200; record displays the new values.
8	POST /enrollments (blank name)	400; “Student name is required.”
9	GET /enrollments/99999	404; “Enrollment not found.”
10	DELETE /enrollments/{id}	204 and an empty response body.
11	GET same deleted ID	404; enrollment no longer exists.

Step 31 — Test invalid input in Swagger

Try submitting invalid registration data to prove the validation code works. We expect a 400 response because this is a dictionary body that reaches our business-rule validator.

JSON — INTENTIONALLY INVALID

```
{
  "student_name": "   ",
  "email": "incorrect-email",
  "phone": "08012345678",
  "course_id": 2
}
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

student_name spaces only: After strip(), the value is empty, so our first validation rule rejects the request.

email without @: This is also invalid in our basic classroom rule, but the student-name error is raised first.

400 response: The route catches ValueError and translates it to a client-error HTTP response.

CHECKPOINT Change student_name to a real value and rerun; you should then see the email error. Fix email too and try an unknown course_id to test the final rule.

Step 32 — Test the complete browser journey

In the browser dashboard: register Ada in course 2; register Bello in course 1; choose course 2 in the filter; click Clear; edit Ada's phone and course; delete Bello. Each step should produce the expected visible change or success message. Note that records disappear on process restart.

KNOWN PROTOTYPE LIMIT This app is intended for one classroom process. It has no persistence, login or CSRF protection; do not operate it with real personal information or rely on consistent in-memory data across multiple deployed worker processes.

PHASE 8 · Git and GitHub

Publishing the source shows that students understand basic collaboration and project handoff. Never upload a virtual environment or private environment settings.

Step 33 — CREATE .gitignore

At the root of the project, create .gitignore and paste this content.

.GITIGNORE — CREATE

```
.venv/
__pycache__/
*.pyc
.env
.vscode/
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

.venv/: Excludes installed packages and local virtual-environment files.

__pycache__/ and *.pyc: Excludes generated Python bytecode and cache folders.

.env: Excludes local secret/configuration files if later introduced.

.vscode/: Keeps personal editor settings out of the class repository.

Step 34 — CREATE README.md

Create README.md at the root. This documentation tells another person how to start and test the project.

README.MD — CREATE

```
# Nascomsoft Course Registration — Classroom Demo

A beginner FastAPI project with a small server-rendered HTML/CSS dashboard.

## Quick start

1. Create a virtual environment: `python -m venv .venv` (use `python3` where needed).
2. Activate it: Windows PowerShell `.venv\Scripts\Activate.ps1`; macOS/Linux `source .venv/bin/activate`.
3. Install dependencies: `python -m pip install -r requirements.txt`.
4. Run: `python -m uvicorn main:app --reload`.
5. Open `http://127.0.0.1:8000/` and `http://127.0.0.1:8000/docs`.

## What you can do

- List training courses and view one course.
- Create, read, update, delete and filter enrollment records.
- Register students and manage enrollments in a simple HTML interface.
- Try the REST API through FastAPI's `/docs` page.

## Deployment note

A Python host such as Render can run the app from a GitHub repo. Build: `pip install -r requirements.txt`. Start: `uvicorn main:app --host 0.0.0.0 --port $PORT`.

## Important limitations

This is an educational prototype. It has no authentication, no permanent storage, and very basic email checking. **Do not enter real students' personal information**. All enrollment data disappears when the app process restarts or is redeployed.
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

Project title and description: Explain what the demo is and who it is for.

Quick start: Tell another developer how to install requirements, run Uvicorn and open /docs.

Features: Summarize the functionality that should actually work.

Deployment note: Distinguish an online Python web host from GitHub source hosting.

Limitations: State clearly that the app is public-demo-only, has no authentication and loses records on restart.

Step 35 — Initialize Git and make the first commit

Stop Uvicorn with Ctrl+C if you want a clean terminal. Run these commands from your project directory.

TERMINAL — GIT

```
git init
git status
git add .
git commit -m "Build Nascomsoft registration demo"
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

git init: Starts a local Git repository in the project folder.

git status: Shows which files Git will track or ignore.

git add .: Stages your project source files for a commit.

git commit -m: Records a named snapshot of the complete working project.

CHECKPOINT If Git asks for your name/email, configure Git identity using your own details, then retry the commit.

Step 36 — Connect and push the repository

Create an empty GitHub repository named nascomsoft-course-registration (do not initialize it with another README if your local one is committed). Replace USERNAME with your own GitHub username.

TERMINAL — GITHUB

```
git branch -M main
git remote add origin https://github.com/USERNAME/nascomsoft-course-registration.git
git push -u origin main
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

git branch -M main: Names the current branch main.

git remote add origin: Connects the local repository to your GitHub URL.

git push -u origin main: Uploads the commit and remembers which remote branch the local branch tracks.

CHECKPOINT Refresh the GitHub repository page. It should contain main.py, data_store.py, templates/, static/, requirements.txt, README.md and .gitignore. It should NOT contain .venv/.

PHASE 9 · Deploy and demonstrate

A GitHub repository stores source code, but does not execute FastAPI. For a working public URL, connect that repository to a Python web-service host. Use fictional data only.

Step 37 — Deploy the repository with a Python web host

On Render, create a new Web Service, connect the GitHub repository, select a supported Python runtime/plan, and set the commands below. Review the plan/price before accepting. The provider then installs dependencies and starts Uvicorn.

RENDER — BUILD COMMAND

```
pip install -r requirements.txt
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

pip install -r requirements.txt: Installs the packages required for FastAPI, Uvicorn, Jinja2 and HTML form support in the hosting environment.

RENDER — START COMMAND

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

uvicorn main:app: Runs the FastAPI object named app inside main.py.

--host 0.0.0.0: Makes the application listen on the container network interface, not only the internal localhost interface.

--port $PORT: Uses the port assigned by the hosting platform; it is not the literal text “$PORT” at runtime in a shell.

CHECKPOINT After deployment, open the host-provided URL, /health, /courses, /docs, and the homepage. Register a fictional student and demonstrate an update/delete. If a deployment restarts, records can disappear by design.

Step 38 — Present the finished application

DEMO MOMENT	WHAT TO EXPLAIN
Open dashboard	This is the Nascomsoft training-administrator interface.
Select a course and register	HTML POST sends data to FastAPI; validation runs; storage receives a dictionary.
View/filter	GET returns enrollment records and an optional course_id filter.
Update/delete	The same CRUD helpers serve REST endpoints and simple browser-form routes.
Open /docs	REST API demonstrates GET, POST, PUT, DELETE and 201/400/404/204 responses.
Show repository and URL	GitHub stores source; Python hosting runs it.

POST-CLASS PRACTICE · Task and worked solution

Each exercise extends the exact same app. First attempt the task independently; then compare with the solution immediately below.

Task A — Add a summary endpoint

Requirement: Add an endpoint GET /summary that tells the administrator how many courses exist and how many students are currently enrolled. Do not use a database.

WORKED ANSWER — APPEND TO MAIN.PY

```
@app.get("/summary")
def enrollment_summary():
    return {
        "total_courses": len(COURSES),
        "total_enrollments": len(list_enrollments()),
    }
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

GET /summary: Defines a read-only endpoint for basic reporting.

len(COURSES): Counts sample course records.

len(list_enrollments()): Counts the current enrollment dictionaries.

return dictionary: FastAPI serializes the counts into JSON. The numbers reset with the in-memory data when the server restarts.

Task B — Introduce a new demo course

Requirement: Add a sixth illustrative course named “Internet of Things Fundamentals”, give it ID 6 and a duration, and make sure it appears on GET /courses and on the registration form.

WORKED ANSWER — ADD INSIDE COURSES LIST

```
{
    "course_id": 6,
    "title": "Internet of Things Fundamentals",
    "duration": "4 Weeks",
    "description": "Connected devices and simple IoT concepts.",
},
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

course_id: 6: Chooses a unique ID not already used by the existing five items.

title / duration / description: Supply the same dictionary keys required by the rest of the project.

Where to paste: Insert the dictionary inside COURSES, before its final closing bracket in data_store.py.

Why both UI and API update: Both use the same COURSES list, so adding one correctly formed dictionary updates GET /courses and the Jinja2 dropdown automatically.

Task C — Make error handling visible

Requirement: Use /docs to test an update on enrollment ID 99999, then explain the returned status code in one sentence.

TEST INPUT — HTTP METHOD + JSON

```
PUT /enrollments/99999

{
  "student_name": "Demo Person",
  "email": "demo@example.com",
  "phone": "0000000000",
  "course_id": 2
}
```

CODE EXPLAINED — READ THIS BEFORE CONTINUING

Expected: 404 Not Found: update_enrollment searches for ID 99999, raises LookupError when absent, and the FastAPI route turns it into an HTTP 404.

Why not 400: The request details may be valid; the problem is that the target resource does not exist.

Troubleshooting · Common classroom blockers

Solve errors in the order shown. Read the terminal message carefully before editing code.

SYMPTOM	LIKELY CAUSE & FIX
ModuleNotFoundError: fastapi	Activate .venv, then run python -m pip install -r requirements.txt.
Form data requires python-multipart	Ensure python-multipart is in requirements and install it.
Could not import module "main"	Run Uvicorn from the project root and verify the filename main.py.
No module named "data_store"	Place data_store.py beside main.py, not inside templates/.
TemplateNotFound: index.html	Create templates/index.html with the folder exactly named templates.

SYMPTOM	LIKELY CAUSE & FIX
404 on /static/styles.css	Create static/styles.css and keep app.mount("/static", ...) in main.py.
Port 8000 already in use	Stop the previous Uvicorn process with Ctrl+C or choose another local port.
405 Method Not Allowed	Check the method: a GET-only URL does not automatically accept POST.
422 from Swagger	Check the URL ID type and JSON body type: Body(...) expects a JSON object.
404 on enrollment	Use an ID that actually exists; records reset on server restart.
GitHub push asks for authentication	Authenticate using GitHub's approved login/personal access token workflow.
Online page loads but records vanish	Expected with in-memory data across restarts/instances; a future database solves persistence.

Final acceptance checklist & references

Use this checklist before submitting or presenting the project to the class.

☐ 01. Project folders and dependencies are correct; virtual environment is activated.

☐ 02. GET /health returns JSON with status ok.

☐ 03. Five sample courses appear on /courses and on the webpage.

☐ 04. POST /enrollments returns 201 for valid fictional data.

☐ 05. GET /enrollments and GET /enrollments/{id} return the expected data.

☐ 06. GET /enrollments?course_id=2 filters registrations.

☐ 07. PUT updates a record without changing its enrollment ID.

☐ 08. DELETE returns 204 and removes the record.

☐ 09. 404 and 400 test cases behave predictably.

☐ 10. The HTML form, edit forms, delete button and CSS all work.

☐ 11. README and .gitignore are present; .venv is not committed.

☐ 12. The repo is on GitHub; local or deployed demo is available.

DEFINITION OF DONE A student can explain the complete path: HTML form or API client → HTTP request → FastAPI route → validation + Python CRUD helper → list of dictionaries → response/redirect → updated browser display.

Further reading

Official FastAPI first steps: https://fastapi.tiangolo.com/tutorial/first-steps/

FastAPI HTML templates: https://fastapi.tiangolo.com/advanced/templates/

FastAPI form fields: https://fastapi.tiangolo.com/tutorial/request-forms/

Render FastAPI deployment guide: https://render.com/docs/deploy-fastapi

Reminder: Python data lists are suitable for this educational prototype, but production training enrollment needs permanent storage, authentication and more comprehensive validation.